Object Oriented Programming (CSEG1044)
Unit 03 – Lecture 03 Notes
Multithreading + Synchronization

Tofik Ali

February 27, 2026

Contents

1 Syllabus Alignment (Unit 03) **1**

2 Prerequisites **2**

3 Learning Outcomes **2**

4 Lecture Overview **2**

5 Key Terms (Quick) **2**

6 Detailed Notes **3**
- 6.1 Concurrency vs parallelism (quick intuition) . . . 3
- 6.2 Threads in Java: `Thread` and `Runnable` . . . 3
- 6.3 Thread lifecycle (simplified) . . . 3
- 6.4 Thread priorities (what they mean and what they do not) . . . 3
- 6.5 The real problem: shared mutable state . . . 3
- 6.6 Synchronization with `synchronized` (core tool) . . . 4
- 6.7 Deadlock (common pitfall) . . . 4

7 Worked Example (tutorial): see a race condition and fix it **4**
- 7.1 Another worked example: bank account withdrawal (classic) . . . 5

8 In-Class Exercises (with brief solutions) **6**

9 Self-Study Practice (with hints) **6**

10 Summary (what to remember) **6**

11 Checkpoint Questions (with sample answers) **7**

1 Syllabus Alignment (Unit 03)

Unit 03: threads, priorities, synchronization

2 Prerequisites

You should already be comfortable with:

- classes/objects and methods,
- loops and basic debugging,
- the idea of shared variables and side-effects.

3 Learning Outcomes

After this lecture, you should be able to:

- Create threads using `Thread` and `Runnable` and explain `start()` vs `run()`.
- Describe the thread lifecycle and understand what thread priority means (and doesn't mean).
- Identify race conditions and fix them using `synchronized`.

4 Lecture Overview

Multithreading allows a Java program to do multiple tasks at the same time (or appear to do so). You need threads when you care about:

- **responsiveness**: UI apps should not freeze while doing I/O or long work,
- **throughput**: servers handle many requests concurrently,
- **parallelism**: CPUs have multiple cores that can run tasks in parallel.

Big warning: threads make programs harder to reason about. Most bugs come from shared mutable data. This lecture focuses on the core ideas required by the syllabus: creating threads, priorities, and synchronization.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

5 Key Terms (Quick)

- **Concurrency**: managing multiple tasks that overlap in time.
- **Parallelism**: tasks literally running at the same time on different cores.
- **Thread**: lightweight unit of execution within a process.
- **Race condition**: outcome depends on timing/interleaving of threads.
- **Critical section**: code that must not be executed by multiple threads at once.
- **Lock/monitor**: mechanism used by `synchronized`.

6 Detailed Notes

6.1 Concurrency vs parallelism (quick intuition)

- **Concurrency**: tasks overlap in time (switching between tasks).
- **Parallelism**: tasks run at the same time on different cores.

You can have concurrency on a single-core CPU (by switching), and you can have parallelism on multi-core CPUs.

6.2 Threads in Java: `Thread` and `Runnable`

Two common ways to create threads:

- extend `Thread` and override `run()`,
- implement `Runnable` and pass it to a `Thread` (preferred in many designs).

Most important rule: `start()` vs `run()`.

- `start()` creates a **new OS-level thread** and then calls `run()` on that new thread.
- `run()` called directly is just a normal method call (no new thread).

Thread naming helps debugging.

```
Thread t = new Thread(task);
t.setName("worker-1");
t.start();
```

6.3 Thread lifecycle (simplified)

- **New**: thread object created, not started.
- **Runnable/Running**: eligible to run or currently running.
- **Blocked/Waiting**: sleeping, waiting for lock, or waiting for I/O.
- **Terminated**: finished `run()`.

6.4 Thread priorities (what they mean and what they do not)

Priority is a hint to the scheduler. It is not a guarantee. On many systems it may have little visible effect. Do not rely on priority for correctness.

6.5 The real problem: shared mutable state

If each thread works on its own local variables, life is easy. Bugs start when threads share and modify the same object/variable.

Race condition. A race condition happens when the outcome depends on timing/interleaving. The classic example is `value++`:

- read `value`,
- add 1,
- write back.

If two threads interleave those steps, an increment can be lost.

6.6 Synchronization with `synchronized` (core tool)

`synchronized` protects a **critical section** using a lock (monitor). Only one thread at a time can execute synchronized code guarded by the same lock.

Two forms you must know.

- synchronized instance method: locks on `this`.
- synchronized block: locks on a specific object (`synchronized(lock){...}`).

Lock scope tip. Keep the synchronized part as small as possible, but large enough to keep shared state consistent.

6.7 Deadlock (common pitfall)

Deadlock occurs when:

- thread A holds lock 1 and waits for lock 2,
- thread B holds lock 2 and waits for lock 1.

Prevention rule: always acquire multiple locks in a consistent order.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

7 Worked Example (tutorial): see a race condition and fix it

We will build a tiny counter and run two threads on it.

Step 1: Unsafe counter (race condition)

```
class Counter {
    private int value = 0;
    void inc() { value++; } // not thread-safe
    int get() { return value; }
}
```

Step 2: Safe counter using `synchronized`

```
class SafeCounter {
    private int value = 0;
    synchronized void inc() { value++; }
    synchronized int get() { return value; }
}
```

Step 3: Run two threads and wait using `join()`

```
public class Demo {
    public static void main(String[] args) throws InterruptedException {
        SafeCounter c = new SafeCounter();

        Runnable task = new Runnable() {
            @Override public void run() {
                for (int i = 0; i < 100000; i++) c.inc();
            }
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println(c.get()); // expected 200000
    }
}
```

Why `join()`? It makes the main thread wait until worker threads finish. Without join, main may print the result too early.

7.1 Another worked example: bank account withdrawal (classic)

This example shows why checks and updates must be atomic (together).

Unsafe version (can go negative or allow double-withdraw)

```
class BankAccount {
    private int balance = 100;

    void withdraw(int amount) {
        if (amount <= 0) throw new IllegalArgumentException("amount");

        if (balance >= amount) {
            // force a context switch to make the bug easier to see
            try { Thread.sleep(1); } catch (InterruptedException ignored) {}
            balance -= amount;
        }
    }

    int getBalance() { return balance; }
}
```

Two threads can both pass the check before either subtracts.

Safe version (synchronized critical section)

```
class SafeBankAccount {
    private int balance = 100;

    synchronized void withdraw(int amount) {
        if (amount <= 0) throw new IllegalArgumentException("amount");
        if (balance >= amount) balance -= amount;
    }

    synchronized int getBalance() { return balance; }
}
```

8 In-Class Exercises (with brief solutions)

1. What is the difference between concurrency and parallelism?

   Answer: concurrency is overlapping tasks; parallelism is simultaneous execution on multiple cores.

2. Which method starts a new thread: `start()` or `run()`?

   Answer: `start()`.

3. Why is `value++` unsafe without synchronization?

   Answer: it's a read-modify-write sequence; interleaving can lose updates.

9 Self-Study Practice (with hints)

1. Write a program that runs two threads printing numbers 1–10 with thread names. **Hint:** use `Thread.currentThread().getName()`.

2. Modify the counter example: remove `synchronized` and run 5 times; record outputs. **Hint:** race bugs may not appear every time.

3. Create a shared bank account and two threads withdrawing money; prevent negative balance using synchronization. **Hint:** the check and update must be inside the same synchronized block/method.

4. Try changing thread priorities and observe (note that results may vary).

10 Summary (what to remember)

- Use `start()` to run code in a new thread; `run()` is just a method call.
- Use `join()` when one thread must wait for another to finish.
- Race conditions happen with shared mutable state.

- `synchronized` protects critical sections using a lock (monitor).
- Priority is not a correctness tool; it is only a hint.

11 Checkpoint Questions (with sample answers)

Q1. What is a race condition?

Sample answer: A race condition happens when multiple threads access shared mutable state without proper synchronization, so the outcome depends on timing/interleaving.

Q2. Why does `synchronized` help?

Sample answer: It ensures only one thread at a time can execute the protected critical section, preventing lost updates and inconsistent states.